

T — Register New Account

TC-REG-001

Title: Register successfully with valid, unique email/password/name

Preconditions:

- User is on the [/register](#) page.
- The email used does not already exist in the system.

Steps:

1. Enter a valid unique email.
2. Enter a valid full name.
3. Enter a valid password.
4. Click "Register".

Expected Result:

- Account is created successfully.
- User is automatically logged in (session/token issued).
- User is redirected to [/onboarding](#).

Priority: High

TC-REG-002

Title: Register with an email that already exists

Preconditions:

- An account with the email already exists in the system.

Steps:

1. Go to [/register](#).
2. Enter the existing email, a valid name, and a valid password.
3. Click "Register".

Expected Result:

- Registration fails with a clear error message.
- No duplicate account is created in the database.

Priority: High

TC-REG-003

Title: Register with an invalid email format

Steps:

1. Go to `/register`.
2. Enter an invalid email.
3. Enter valid name and password.
4. Click "Register".

Expected Result:

- Client-side and/or server-side validation blocks submission.
- Error message clearly indicates invalid email format.

Priority: Medium

TC-REG-004

Title: Register with a weak/too-short password

Steps:

1. Go to `/register`.
2. Enter valid email and name.
3. Enter a password below the minimum required length/complexity.
4. Click "Register".

Expected Result:

- Registration is rejected with a validation error explaining the password requirement.

Priority: Medium

TC-REG-005

Title: Register with empty required fields

Steps:

1. Go to `/register`.
2. Leave name, email, and/or password blank.
3. Click "Register".

Expected Result:

- Form does not submit; each empty required field shows a validation message.

Priority: Medium

TC-REG-006

Title: Register with special characters / unicode in name and password

Steps:

1. Go to `/register`.
2. Enter a name containing unicode characters.
3. Enter a password containing special characters.
4. Click "Register".

Expected Result:

- Registration succeeds; name and password are stored/handled correctly without corruption.

Priority: Low

TC-REG-007

Title: Verify password is stored hashed with bcrypt (not plaintext)

Preconditions:

- Access to the database or backend logs.

Steps:

1. Register a new account with a known password.
2. Query the `User` table for the newly created row.
3. Inspect the stored password value.

Expected Result:

- Stored value is a bcrypt hash (starts with `$2a$/$2b$`, ~60 chars), never the plaintext password.
- Logging in with the original plaintext password still succeeds (bcrypt.compare works).

Priority: High

TC-REG-008

Title: Auto-login and full onboarding flow after successful registration

Steps:

1. Complete registration with valid data.
2. Observe that user is auto-logged in and redirected to `/onboarding` (no separate login step required).
3. Complete the onboarding flow: set weekly study commitment (Step 1), then take the Placement Test / self-select CEFR level (Step 2).
4. Observe the app after finishing the test/level selection.

Expected Result:

- No separate login step is required; user session is established immediately after registration.
- User is redirected to `/onboarding` (since `hasCompletedOnboarding` is false for a new user).

- After completing the onboarding test, user is taken to a Result page showing their score/assigned CEFR level.
- On the Result page, user is given the option to proceed to [/dashboard](#) (not auto-redirected).
- Clicking that option successfully navigates the user to [/dashboard](#).

Priority: High

TC-REG-009

Title: Register with SQL Injection / script injection payload

Steps:

1. Go to [/register](#).
2. Enter a payload such as ' 0R '1'='1 or [<script>alert\(1\)</script>](#) in the name/email field.
3. Click "Register".

Expected Result:

- Input is either rejected by validation or safely stored/escaped (via Sequelize parameterized queries).
- No SQL error, no script execution, no injection occurs.

Priority: High

TC-REG-010

Title: Register with excessively long input values

Steps:

1. Go to [/register](#).
2. Enter a name/email of 1000+ characters.
3. Click "Register".

Expected Result:

- System either enforces a max-length validation error or truncates/handles gracefully without crashing (no 500 error).

Priority: Low

UC2 — Login

TC-LOGIN-001

Title: Login successfully with correct credentials

Preconditions:

- A registered account exists with known email/password.

Steps:

1. Go to `/login`.
2. Enter the correct email and password.
3. Click "Login".

Expected Result:

- Login succeeds (HTTP 2xx).
- `accessToken` (15 min expiry) and `refreshToken` (7 day expiry) are issued.
- User is redirected to `/dashboard` (if onboarded) or `/onboarding` (if not).

Priority: High

TC-LOGIN-002

Title: Login with correct email but wrong password

Steps:

1. Go to `/login`.
2. Enter a valid registered email with an incorrect password.
3. Click "Login".

Expected Result:

- Login fails with HTTP 401.
- Generic error message shown (does not reveal whether email exists).

Priority: High

TC-LOGIN-003

Title: Login with a non-existent email

Steps:

1. Go to `/login`.
2. Enter an email that has never been registered.
3. Enter any password.
4. Click "Login".

Expected Result:

- Login fails with HTTP 401.
- Error message is identical/generic to TC-LOGIN-002 (does not reveal account existence).

Priority: High

TC-LOGIN-004

Title: Login with empty email/password fields

Steps:

1. Go to `/login`.
2. Leave email and/or password blank.
3. Click "Login".

Expected Result:

- Client-side validation blocks submission; no request sent to backend.

Priority: Medium

TC-LOGIN-005

Title: Timing attack mitigation check (wrong password vs non-existent email)

Preconditions:

- Ability to measure response times (browser DevTools Network tab or a script).

Steps:

1. Send a login request with a valid email + wrong password; record response time.
2. Send a login request with a non-existent email + any password; record response time.
3. Compare the two response times over multiple attempts.

Expected Result:

- Response times are similar (no significant/consistent gap that would let an attacker distinguish "user exists" from "user doesn't exist").

Priority: Medium

TC-LOGIN-006

Title: Account lockout after 5 consecutive failed login attempts

Preconditions:

- A registered account exists.

Steps:

1. Attempt to log in with the correct email and a wrong password 5 times in a row.
2. Attempt a 6th login (with either correct or incorrect password).

Expected Result:

- On the 6th attempt, the system blocks the login attempt, even if credentials are correct.

Priority: High

TC-LOGIN-007

Title: Login succeeds again after lockout period expires

Preconditions:

- Account was locked out per TC-LOGIN-006.

Steps:

1. Wait for the lockout period to expire (per system policy).
2. Log in again with correct credentials.

Expected Result:

- Login succeeds normally once the lockout window has passed.

Priority: Medium

TC-LOGIN-008

Title: Login with different email casing

Steps:

1. Register with email `Test@Mail.com`.
2. Attempt login using `test@mail.com` (lowercase).

Expected Result:

- Document actual behavior: login succeeds if email lookup is case-insensitive, or fails with a clear error if case-sensitive is the intended design.

Priority: Low

TC-LOGIN-009

Title: Access token expires after 15 minutes

Preconditions:

- User is logged in and holds a valid `accessToken`.

Steps:

1. Wait 15+ minutes without renewing the token (or manually manipulate token expiry for testing).
2. Call a protected API using the expired token.

Expected Result:

- Request is rejected with HTTP 401 (token expired).

Priority: Medium

TC-LOGIN-010

Title: Correct post-login redirect based on onboarding status

Steps:

1. Log in with an account that has NOT completed onboarding.
2. Log out, then log in with an account that HAS completed onboarding.

Expected Result:

- Case 1: redirected to `/onboarding`.
- Case 2: redirected to `/dashboard`.

Priority: High

UC6 — Remember Login

TC-REM-001

Title: Verify token storage mechanism after login

Steps:

1. Log in successfully.
2. Inspect browser storage (Application tab: Local Storage / Cookies).

Expected Result:

- Document actual storage location of `accessToken/refreshToken` and confirm it matches the intended security design.

Priority: High

TC-REM-002

Title: Token is not exposed to client-side JavaScript if HttpOnly is expected

Steps:

1. Log in successfully.
2. In the browser console, run `document.cookie` and inspect `localStorage`.

Expected Result:

- If design requires HttpOnly cookies, tokens must NOT appear in `document.cookie` or be readable via JS.

- Record actual result if current implementation stores tokens in localStorage instead (potential security gap to flag).

Priority: High

TC-REM-003

Title: Session persists after closing and reopening the browser tab

Steps:

1. Log in successfully.
2. Close the browser tab (do not log out).
3. Reopen the app in a new tab.

Expected Result:

- User remains logged in without needing to re-enter credentials.

Priority: High

TC-REM-004

Title: Session persists after page refresh (F5)

Steps:

1. Log in successfully and navigate to `/dashboard`.
2. Refresh the page (F5).

Expected Result:

- User remains on `/dashboard`, still authenticated; no forced redirect to `/login`.

Priority: High

TC-REM-005

Title: Auto-refresh of access token after expiry

Preconditions:

- User is logged in ("Remember Login" active).

Steps:

1. Wait until `accessToken` expires (15 minutes) or simulate expiry.
2. Perform an action requiring a protected API call.

Expected Result:

- System silently uses the **refreshToken** to obtain a new **accessToken** without forcing the user to log in again.

Priority: High

TC-REM-006

Title: Refresh token rotation invalidates the old refresh token

Steps:

1. Trigger a token refresh (per TC-REM-005).
2. Capture the old and new **refreshToken** values.
3. Attempt to use the OLD **refreshToken** to request another refresh.

Expected Result:

- A new **refreshToken** is issued on each refresh.
- The old **refreshToken** is rejected (invalidated) when reused.

Priority: Medium

TC-REM-007

Title: Logout clears stored tokens

Steps:

1. Log in successfully.
2. Click "Logout".
3. Inspect browser storage (localStorage/cookies).

Expected Result:

- **accessToken** and **refreshToken** are removed from storage.
- User is redirected to **/login**.

Priority: High

TC-REM-008

Title: Old token cannot be reused after logout

Steps:

1. Log in and capture the **accessToken**.
2. Log out.
3. Manually replay a protected API call using the captured (now-stale) **accessToken**.

Expected Result:

- Request is rejected with HTTP 401.

Priority: High

TC-REM-009

Title: Tampered/invalid token forces re-login

Steps:

1. Log in successfully.
2. Manually edit the stored `accessToken` value in browser storage (corrupt it).
3. Refresh the page or trigger a protected API call.

Expected Result:

- System detects the invalid token, clears the session, and redirects to `/login`.

Priority: Medium

TC-REM-010

Title: Concurrent login sessions on two different browsers/devices

Steps:

1. Log in with the same account on Browser A.
2. Log in with the same account on Browser B (different browser or incognito).
3. Perform actions on both sessions.

Expected Result:

- Document actual behavior: both sessions work independently, or the system restricts to a single active session — confirm which matches the intended design.

Priority: Low

Test Execution

Test Case ID	Execution Date	Status	Actual Result	Bug ID
TC-REG-001	2026-08-20	Pass	Account created, user auto-logged in and redirected to <code>/onboarding</code> as expected.	
TC-REG-002	2026-08-20	Pass	Registration rejected with "Email already in use" error; no duplicate row created.	

Test Case ID	Execution Date	Status	Actual Result	Bug ID
TC-REG-003	2026-08-20	Pass	Invalid email format blocked with clear validation error.	
TC-REG-004	2026-08-20	Fail	Weak/too-short password was accepted and the account was created successfully; no password-policy validation error was shown.	BUG-001
TC-REG-005	2026-08-20	Pass	Form did not submit; validation messages shown for each empty required field.	
TC-REG-006	2026-08-20	Pass	Registration succeeded; unicode name and special-character password stored/handled correctly.	
TC-REG-007	2026-08-20	Pass	Stored password value is a bcrypt hash (\$2a\$/\$2b\$ prefix); login with original plaintext password still succeeds.	
TC-REG-008	2026-08-20	Pass	Auto-login after registration works; onboarding flow completes and Result page offers navigation to /dashboard as expected.	
TC-REG-009	2026-08-20	Pass	Injection payload safely stored/escaped via Sequelize; no SQL error or script execution.	
TC-REG-010	2026-08-20	Pass	Excessively long input handled gracefully (validation error / no server crash).	
TC-LOGIN-001	2026-08-20	Pass	Login succeeded; accessToken and refreshToken issued; redirected correctly based on onboarding status.	
TC-LOGIN-002	2026-08-20	Pass	Login rejected with HTTP 401 and generic error message.	
TC-LOGIN-003	2026-08-20	Pass	Login rejected with HTTP 401; error message identical/generic to TC-LOGIN-002.	
TC-LOGIN-004	2026-08-20	Pass	Client-side validation blocked submission with empty fields; no request sent to backend.	

Test Case ID	Execution Date	Status	Actual Result	Bug ID
TC-LOGIN-005	2026-08-20	Pass	Response times for wrong-password vs non-existent-email were comparable; no distinguishable timing gap.	
TC-LOGIN-006	2026-08-20	Fail	No account lockout mechanism exists; the 6th (and subsequent) login attempts are processed normally with no rate-limiting/lockout response.	BUG-002
TC-LOGIN-007	2026-08-20	N/A	Not executed independently — lockout feature does not exist (see TC-LOGIN-006 / BUG-002), so this case cannot be verified.	BUG-002
TC-LOGIN-008	2026-08-20	Pass	Email lookup is case-insensitive; login with different casing succeeds as intended.	
TC-LOGIN-009	2026-08-20	Fail	AccessToken is still valid after 15 minutes; the system actually issues an accessToken with a 1-day expiry rather than the documented 15-minute expiry, so the protected API call still returned success instead of HTTP 401.	BUG-003
TC-LOGIN-010	2026-08-20	Pass	Case 1 redirected to /onboarding ; Case 2 redirected to /dashboard , as expected.	
TC-REM-001	2026-08-20	Pass	Token storage location documented and matches intended design.	
TC-REM-002	2026-08-20	Pass	Token exposure behavior confirmed to match intended design.	
TC-REM-003	2026-08-20	Pass	Session persisted after closing/reopening the tab; user stayed logged in.	
TC-REM-004	2026-08-20	Pass	Session persisted after page refresh (F5); no forced redirect to /login .	
TC-REM-005	2026-08-20	Pass	Access token auto-refreshed silently via refreshToken without forcing re-login.	
TC-REM-006	2026-08-20	Pass	New refreshToken issued on refresh; old refreshToken rejected when reused.	

Test Case ID	Execution Date	Status	Actual Result	Bug ID
TC-REM-007	2026-08-20	Pass	Logout cleared stored tokens; user redirected to /login .	
TC-REM-008	2026-08-20	Pass	Replaying the stale accessToken after logout was rejected with HTTP 401.	
TC-REM-009	2026-08-20	Pass	Tampered token detected; session cleared and user redirected to /login .	
TC-REM-010	2026-08-20	Pass	Concurrent sessions on two browsers behaved independently, matching intended design.	